

Technical Reference Guide

How to explain the main files, functions, and data flow

How to Present the Project

Explain the system as a sequence: discover files, extract text, build a Candidate model, apply eligibility, score eligible candidates, enrich GitHub, sort by score, and write JSON. Emphasize that eligibility is deterministic and happens before ranking.

Main Files

File	Purpose and explanation
main.py	CLI entry point. Parses --input, --output, and --no-github arguments, creates the GitHub client, runs the pipeline, and prints a concise summary.
src/parser.py	Format-aware ingestion. discover_resume_files finds PDF, DOCX, and TXT files. extract_pdf_text uses pypdf and reads PDF link annotations. extract_docx_text reads paragraphs from DOCX XML. extract_text_resume handles UTF-8 TXT. extract_resume_text dispatches by extension.
src/models.py	Pydantic contracts. Candidate stores extracted resume data; Project stores project details; EligibilityResult stores hard-filter evidence; ScoreResult stores category scores and penalties; GitHubResult stores enrichment status.
src/utils.py	Conservative extraction helpers. normalize and section helpers clean layout artifacts. _skills detects explicit and canonical technologies. _github_url selects profile URLs. extract_candidate builds the structured Candidate without inventing missing facts.
src/eligibility.py	Owns the hard filter. detect_python_evidence and detect_ai_evidence collect evidence. evaluate_eligibility returns eligible=True only when both evidence groups exist.
src/scorer.py	Owns explainable ranking. It calculates the five weighted categories, caps each category, applies visible shallow-wrapper penalties, and raises an error if an ineligible candidate is passed.
src/github.py	Optional public enrichment. Extracts username, caches per username, calls profile/repository/event endpoints, estimates recent activity and relevance, and converts failures into structured statuses.
src/llm.py	Provider boundary. An optional structured extractor can be injected. If it fails, deterministic extraction is used and the failure is recorded.
src/pipeline.py	Orchestrator. Hashes files for duplicates, parses each file independently, extracts candidates, evaluates eligibility, enriches eligible candidates, scores them, assigns rank and ranking_number, and writes results.json.
tests/	Focused tests cover parser formats, hyperlink extraction, name handling, eligibility, scoring, GitHub caching/failures, and batch failure isolation.

Important Data Structures

Candidate

Represents one resume after extraction: name, contact data, skills, projects, experience, Python evidence, AI evidence, engineering evidence, and GitHub URL.

EligibilityResult

Represents the hard decision and the exact evidence or rejection reasons. This object is created before any score is calculated.

ScoreResult

Represents the total score, category breakdown, evidence lists, and visible penalties. The maximum is 100.

GitHubResult

Represents profile URL, username, status, public activity summary, score, and error details. GitHub failure never rejects a candidate.

End-to-End Explanation

- 1. `discover_resume_files` recursively finds supported files.
- 2. `extract_resume_text` selects the correct parser and normalizes text.
- 3. `extract_candidate` creates a Pydantic Candidate using evidence only.
- 4. `evaluate_eligibility` applies Python plus AI hard requirements.
- 5. GitHub enrichment runs only for eligible candidates and is cached.
- 6. `score_candidate` calculates the explainable 100-point score.
- 7. pipeline sorts eligible candidates descending and assigns `rank/ranking_number`.
- 8. JSON output includes eligible, rejected, failed, duplicate, evidence, and batch summary sections.

Interview Talking Points

- Why deterministic eligibility? It is predictable, testable, and prevents ranking from overriding minimum requirements.
- Why a fallback extractor? The batch remains useful without an LLM key and continues when an optional provider fails.
- Why cache GitHub? It avoids repeated network calls for the same public profile.
- Why visible penalties? They make shallow AI projects explainable instead of silently reducing scores.
- Why isolate files? One malformed resume must not terminate processing for the other candidates.